

Vehicular Intrusion Detection System Based on Hybrid Quantum Neural Networks

Jianhua Zhou¹, Yueheng Liu², Sifan Li¹, Yue Cao^{1*}, Wenkai Chen³, Naveed Ahmed⁴

¹School of Cyber Science and Engineering, Wuhan University, China. yue.cao@whu.edu.cn

²Wuhan Cyber Security Association, China.

³Ecarx (Hubei) Technology Co., Ltd, China.

⁴College of Computer and Information Sciences, Prince Sultan University, Riyadh, Saudi Arabia.

Abstract—Traditional Deep Learning (DL) method is increasingly used in vehicular Intrusion Detection Systems (IDSs). However, there are still some limitations. It combines various models, resulting in a massive model size and numerous parameters, requiring more computational resources. Furthermore, real-time performance is crucial in vehicular IDS. Large models typically require more time for detection, failing to meet the timeliness requirements. Quantum computing offers parallel computing capabilities, prompting our proposal of a Hybrid Quantum Neural Network (HQNN)-based IDS to improve the timeliness of intrusion detection. This IDS employs HQNN to enhance feature extraction and speed up traditional convolution. It retains most of the structure of classical Convolutional Neural Network (CNN), comprising attention layers, batch normalization layers, and quantum circuits. Through quantum superposition and entanglement, certain complex non-linear functions are compressed. Furthermore, residual connections enable seamless gradient flow during back propagation. This accelerates computation and expedites convergence. Comparative experiments on the Carhacking dataset show improved detection time and convergence speed, with exceptionally high accuracy in practical applications.

Index Terms—Controller area network (CAN), intrusion detection system (IDS), vehicular network, automotive security

I. INTRODUCTION

In vehicular Intrusion Detection System (IDS), Machine Learning (ML) and Deep Learning (DL) are commonly used for classification. These methods often convert data into 2D images for anomaly detection. For example, the Deep Convolutional Neural Network (DCNN) method converts CAN bus data into 2D images for classification [1]. However, as data volumes increase and ML models become more complex, computational resources become a major obstacle for traditional ML algorithms. It can potentially lead to slow convergence rates and may even cause algorithm failures [2]. In comparison, quantum computing holds promise for overcoming this challenge by efficiently addressing the computational resource problem, which is prohibitively expensive for classical computers.

Quantum Machine Learning (QML) integrates classical ML with Quantum Information Processing (QIP), forming

a new field in quantum realm to address issues presented in classical ML. In recent years, researchers have been exploring its potential, making contributions to supplement ML deficiencies in areas like data processing, classification and more. In general, QML algorithms can be categorized into 3 types: QML, quantum-inspired ML, and hybrid quantum-classical ML. QML applies traditional ML to quantum devices. Quantum-inspired ML leverages quantum computing principles to enhance traditional ML algorithms. Hybrid quantum-classical ML combines quantum algorithms with classical machine learning to reduce learning costs.

Quantum Neural Networks (QNNs) are a promising implementation of hybrid quantum-classical ML and have potential in classification domain. Yang and Zhang proposed a quantum DL approach, encoding data into quantum state probability amplitudes. It achieved exponential acceleration in loss function computation using QNN for numerical simulations on the Iris dataset [3]. Cong et al. constructed Quantum Convolutional Neural Networks (QCNN), demonstrating its classification effectiveness in identifying quantum states related to 1D symmetric topological phases [4]. Chen et al. presented two scale-inspired local feature extraction methods based on QCNN, achieving higher accuracy than classical CNNs in binary image classification [5]. Li et al. studied Quantum Deep Convolutional Neural Networks (QDCNN), showing higher accuracy than CNNs on MNIST and GTSRB datasets, providing a new avenue for image classification [6]. Henderson et al. introduced a novel transformation layer - quantum convolutional layer - which outperformed CNN and nonlinear CNN in accuracy and efficiency on the MNIST dataset [7]. Abdel-Khale et al. proposed Quantum Neural Network Based Multi-Labeled Aerial Image Classification (QNN-MLAIC), demonstrating its feasibility and effectiveness in aerial image classification [8].

Researchers have developed various QNN models, such as quantum-inspired neural networks, quantum perceptron neural networks and quantum convolutional neural networks, for real-world applications. Jeswal et al. discussed various QNNs and their applications in real-world problems, highlighting their superiority and improved computational efficiency over classical neural networks [9]. It justifies QNN is particularly suitable for image classification because quantum computing accelerates

This work is supported by Wuhan Industrial base Innovation Program (2023010402010783) and EU HORIZON-TMA-MSCA-SE project TRACE-V2X under grant (101131204).

complex image understanding calculations, providing simple yet decisive output results.

Inspired by QML, we develop a vehicular IDS based on Hybrid Quantum Neural Network (HQNN). It is a model combining classical CNN with quantum networks. By retaining most of the classical CNN architecture while leveraging quantum circuits, it not only extracts information and processes sequential data effectively, but also reduces overall model computation time. The quantum component is a Parameterized Quantum Circuit (PQC) to extract basic features from images, replacing certain convolutional layers. The classical fully connected network handles classification, and classical optimizers adjust parameters of both quantum and classical networks. Additionally, the model utilizes amplitude encoding for image classification tasks, requiring fewer quantum bits compared to position encoding.

II. BACKGROUND

A. Quantum Bit and Quantum State

A quantum bit (qubit) is the smallest unit in quantum information processing. It can exist in a superposition of both 0 and 1 simultaneously. Quantum states, represented as a linear combination of basis states, are expressed as $|y\rangle = \alpha_0|0\rangle + \beta_0|1\rangle = (\alpha_0, \beta_0)^T$, where α_0 and β_0 are complex numbers. They satisfy $|\alpha_0|^2 + |\beta_0|^2 = 1$, known as probability amplitudes. When a qubit is in the state $|y\rangle$, it transitions to states $|0\rangle$ and $|1\rangle$ with probabilities $|\alpha_0|^2$ and $|\beta_0|^2$, respectively.

B. Linear Superposition

Given a set of basic quantum states $|i\rangle$, a quantum state $|\psi\rangle$ can be represented as a linear combination of these states: $|\psi\rangle = \sum_i P_i|i\rangle$, where P_i is the probability amplitude corresponding to the basic state $|i\rangle$. When measuring this quantum state, the system will be observed to be in the basic state $|i\rangle$ with a probability of $|P_i|^2$.

C. Unitary Transformation

In quantum mechanics, transformations of quantum states are represented by matrix multiplication, with operator [10] describing this transition. This operator, denoted as U , is a matrix acting on quantum states with dimensions of $2^N \times 2^N$ (where N is the number of qubits). Such operator is termed unitary matrix, and the transformation is known as a unitary transformation. For instance, if the current quantum state $|y\rangle = |1\rangle$ and the unitary matrix $U = I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, then the resulting quantum state is given as

$$|z\rangle = U|1\rangle = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}. \quad (1)$$

D. Quantum Rotation Gate

Quantum gates are unitary linear transformations of quantum state vectors, each corresponding to a unitary operator U . These gates transition a state $|y_0\rangle$ to another state $|y_1\rangle$, denoted as $|y_1\rangle = U|y_0\rangle$, implying that the action of a quantum gate is

akin to left-multiplying the state vector by its corresponding unitary matrix. Here, we focus on commonly used gates like rotation gates and Controlled-NOT (CNOT) gates in QNN.

The most famous type of rotation gates is y -axis rotation gate $R_y(\theta)$, and its unitary matrix is

$$R_y(\theta) = \begin{bmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{bmatrix}. \quad (2)$$

In fact, this single-qubit gate can be visualized as a transformation on the Bloch sphere. The effect of $R_y(\theta)$ corresponds to rotating the quantum state around the y -axis on the Bloch sphere by an angle θ . The structure of the Bloch sphere is depicted in Fig.1, with $|0\rangle$ and $|1\rangle$ representing a pair of orthogonal bases located along the y -axis. The operation of $R_y(\theta)$ is depicted in Fig.2.

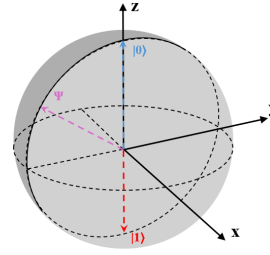


Fig. 1: Bloch sphere.

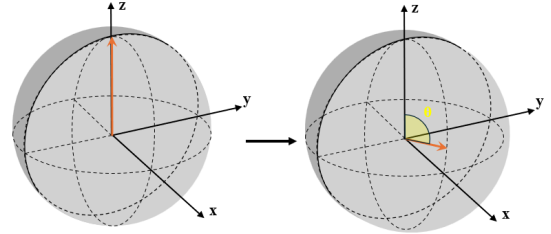


Fig. 2: The effect of $R_y(\theta)$ rotation gate on Bloch sphere.

E. Controlled-Not (CNOT) Gate

CNOT gate is an important example of a two-qubit gate. It operates on two qubits, as depicted in Fig.3. If the first qubit is in the state $|1\rangle$, it flips the state of the second qubit. If the first qubit is in the state $|0\rangle$, the state of the second qubit remains unchanged. Its unitary matrix representation is

$$CNOT = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}. \quad (3)$$

F. Quantum Encoding

In quantum computing, classical information needs to be encoded into quantum states for processing. This encoding process is accomplished using a series of quantum gates. There are several methods to embed classical data into quantum systems as following:

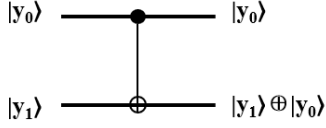


Fig. 3: CNOT gate circuit.

- **Basis encoding:** It involves mapping classical data into the basic states of qubits. In basis encoding, classical data is usually represented in binary form, and then each bit value is encoded into the corresponding qubit. For example, if we have classical data “1010”, the corresponding quantum state may be $|1010\rangle$, where each qubit represents a binary digit.
- **Amplitude encoding:** It is a technique for encoding classical data into the amplitudes of a quantum state. Given a classical vector $\mathbf{x} = (x_1, x_2, x_3, \dots, x_n)$, first we normalize it to obtain a normalized vector $\mathbf{x}' = (x'_1, x'_2, x'_3, \dots, x'_n)$, making $\sum_{i=1}^n |x'_i|^2 = 1$ to ensure the resulting quantum state's amplitudes are also normalized. Next, we map the elements of $\mathbf{x}'$ in sequence to the amplitudes of a quantum state. Specifically, we construct a quantum state $|\psi\rangle$, where the amplitudes correspond one-to-one with those of $\mathbf{x}'$. Thus, for state $|i\rangle$, its amplitude is x'_i . This amplitude encoded quantum state is represented as $|\psi\rangle = \sum_{i=1}^n x'_i |i\rangle$.

G. Quantum Measurement

To extract information, quantum states must be observed and transformed into classical states. The probability of measuring a given state is equal to the square of the probability amplitude associated with that state. For example, if the current quantum state is $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, then the probability of measuring the system in state $|0\rangle$ is $|\alpha|^2$, while in state $|1\rangle$ is $|\beta|^2$. Quantum measurement is represented by a set of measurement operators $\{M_m\}_{m=1}^n$, where the subscript m represents the measurement outcome. Assuming the system is in state $|\psi\rangle$ before measurement, the probability of observing result m after measurement is $\langle\psi|M_m^\dagger M_m|\psi\rangle$, and the state of the system after measurement, given result m , is transformed to

$$|\psi\rangle \rightarrow \frac{M_m|\psi\rangle}{\sqrt{\langle\psi|M_m^\dagger M_m|\psi\rangle}}, \quad (4)$$

where $M_m^\dagger$ is complex conjugate transpose matrix of M_m .

III. PROPOSED FRAMEWORK

A. Quantum Neural Network Implementation

QNN implementation with classical neural network integration requires 3 layers: an encoding layer U_{Enc} for mapping and embedding classical network outputs into the quantum system, a PQC layer U_{PQC} for training and learning, consisting of fixed quantum gates with trainable parameters, and a measurement layer U_{Mea} for measuring quantum states and outputting observable results for classical network use. Its structure is depicted in Fig.4.

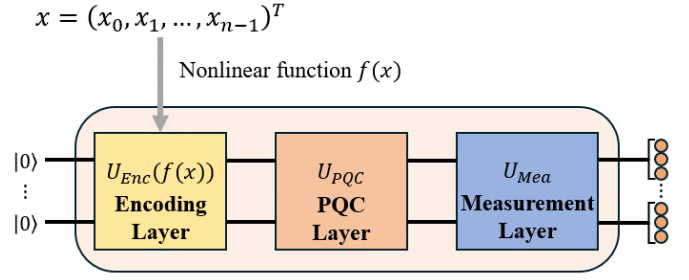


Fig. 4: Quantum neural network architecture diagram.

1) **Encoding Layer:** In the encoding layer U_{Enc} , a nonlinear transformation $f(x)$ is first applied. It maps the classical network output data $x = (x_0, x_1, \dots, x_{n-1})$ to angle space. Subsequently, amplitude encoding is applied to the initial state to convert it into the quantum state $|\psi_{in}\rangle$ input to U_{PQC} . Specifically, the nonlinear function $f(x_i) = \arcsin(x_i)$ is chosen to map the n elements of x to the interval $[-\frac{\pi}{2}, \frac{\pi}{2}]$. Then, $R_y(f(x_i))$ is used to rotate the computational basis state $|0\rangle$ after mapping along the y -axis. That is, $U_{Enc} = R_y(f(x_i))|0\rangle$, where $R_y(\theta)|0\rangle = \cos \frac{\theta}{2}|0\rangle + \sin \frac{\theta}{2}|1\rangle$, and the quantum state of each qubit after rotation is given by

$$|\psi_{in}^i\rangle = \begin{bmatrix} \cos \frac{f(x_i)}{2} \\ \sin \frac{f(x_i)}{2} \end{bmatrix}. \quad (5)$$

Here, we use 4 qubits to encode classical data, meaning $n = 4$.

2) **PQC Layer:** In the PQC layer U_{PQC} , a series of PQC is utilized to perform linear transformations. Generally, a PQC for n qubits can be written in the following form:

$$U_{PQC}(\theta) |\psi_{in}\rangle = \left(\prod_{i=0}^{m-1} U_i(\theta_i) \right) |\psi_{in}\rangle. \quad (6)$$

Here, $U_{PQC}(\theta)$ represents a series of quantum gates with parameters θ , where $\theta = (\theta_0, \theta_1, \dots, \theta_{m-1})$ are the parameters of m rotation gates. In this context, to facilitate training and evaluation, only CNOT gates and $R_y(\theta)$ are employed in the PQC. The circuit structure of U_{Enc} and U_{PQC} is shown in Fig.5. Each qubit first undergoes $R_y(\theta)$, followed by a CNOT gate loop for superimposing quantum states. The CNOT gate loop connects each qubit with its neighbors, and the last qubit is connected to the first qubit. Additionally, influenced by the deep neural network concept, 2 layers of PQC are adopted, meaning two identical quantum circuits are concatenated.

3) **Measurement Layer:** In the measurement layer U_{Mea} , appropriate measurement strategies are employed to extract classical information. In most prior works, such as those described in reference [11], the results of quantum circuits are obtained from the expectation values of Pauli Z operators for one or two qubits. Here, to fully extract hidden information from disentangled quantum states, the expectation values of Pauli Z operators on all qubits are computed. The expected value observed on the k -th qubit based on the measurement operator M is represented as:

$$E(m_k) = \langle\psi_{in}|U_{PQC}^\dagger M U_{PQC}|\psi_{in}\rangle, \quad (7)$$

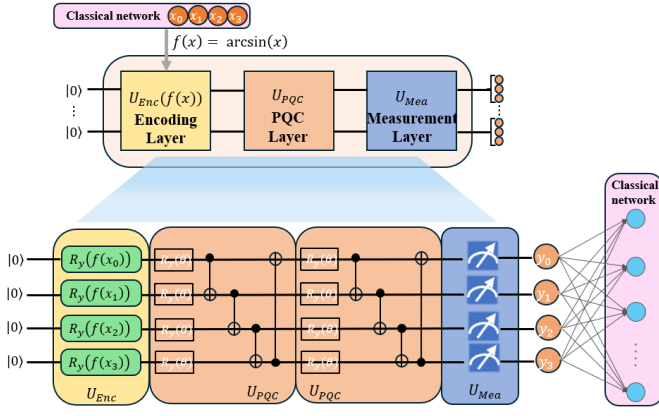


Fig. 5: The circuit composition of quantum neural network.

where $E(m_k)$ represents the observed expectation value of the k -th qubit, ψ_{in} denotes the quantum state after the encoding layer, and $M = \sigma_z$ is the Pauli Z operator, yielding expectation values $E(m_k)$ within the range of $[-1, 1]$.

B. Hybrid Quantum Neural Network Implementation

The proposed model incorporates residual connections, so as to enhance network convergence speed, ease of training and reduce overfitting. Initially, in the main connection, data undergoes several residual modules which are called “Block” comprising QNN layers, attention layers, convolutional layers, pooling layers, and activation functions for feature extraction. Subsequently, in the skip connection, the output of the residual modules is added with primary input to improve model generalization and help alleviate the vanishing gradient problem. The residual module Block is illustrated in Fig.6, and the architecture of the proposed model is depicted in Fig.7.

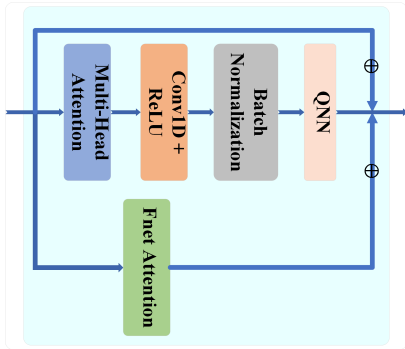


Fig. 6: The network structure of a Block.

The detailed algorithmic workflow of the model is summarized as follows:

- 1) The input data undergoes preprocessing, where features are encoded using one-hot encoding, resulting in a 49-dimensional representation. A time window $T = 64$ is applied, yielding a final 2D tensor of size 64×49 . Subsequently, the data passes through fully connected layers to obtain a 64×4 tensor. This step aims to extract

essential features and reduce dimensionality to better suit subsequent analysis.

- 2) The data undergoes processing through a series of residual modules (referred to as “Block”), each incorporating residual connections with its own output. The internal neural network structure of Block is described sequentially:
 - a) The data (denoted as *Input*) first undergoes a multi-head attention layer which consists of 2 heads. The benefit of this layer lies in its ability to capture complex relationships within input sequences, reflecting the relevance of each position’s input to other parts of the sequence, thereby enhancing the model’s performance and generalization capabilities.
 - b) Subsequently, the data passes through 1D convolutional layer (Conv1D). It employs a convolutional kernel with size of 3, stride of 1, and padding of 2, followed by a ReLU activation function to enhance nonlinear fitting capabilities. While maintaining the dimensions of the output feature maps, it performs convolution operations along the row direction of the input data via sliding windows, capturing crucial local features.
 - c) Next, the data undergoes batch normalization layer. By normalizing the input data of each batch, every layer of the neural network converges faster.
 - d) Then the data is further fed into the QNN layer. After batch normalization layer, the data is represented as $\mathbf{x} = \{\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(64)}\}$, a 2D tensor with a time window of 64, where $\mathbf{x}^{(i)} = \{x_0, x_1, x_2, x_3\}$ represents the feature output at the i -th time step. Firstly, the classical data vector undergoes encoding via $U_{Enc} = R_y(f(x_i))$. Then, a series of linear transformations are applied in U_{PQC} . Finally, the final state is measured to yield classical data $\mathbf{y} = \sigma_z(U_{PQC}|\psi_{in})$, with the same dimensions as the input to the QNN.
 - e) Lastly, the original data *Input* undergoes an Fnet attention layer, residually connect with both the original data and the QNN output $\mathbf{y}$, denoted as $Output = Fnet(Input) \oplus \mathbf{y} \oplus Input$.
- 3) To enhance the model’s depth feature extraction capabilities, the main connection employs consecutive 3 Block modules, with the skip connection being the reduced-dimensional input data.
- 4) The data undergoes a flatten layer, converting the 2D tensor into a 1D vector for input into the fully connected layers. The length of the vector after this layer is 256.
- 5) The data is then passed through a fully connected layer for dimension reduction, with ReLU activation. The output of this layer is a 128-dimensional vector.
- 6) The data continues through a Dropout layer, minimizing the risk of over-fitting.
- 7) Finally, the data undergoes a fully connected layer to get the probability distribution of category labels. The classification in this experiment is binary, distinguishing between “normal” and “attack”, resulting in a 2D vector

representing class labels.

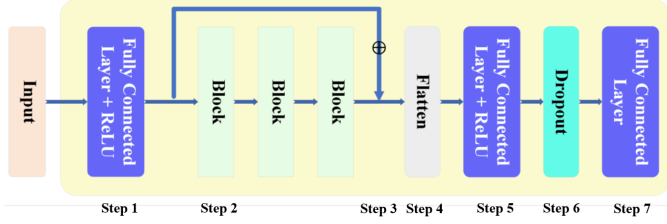


Fig. 7: The network structure of proposed IDS based on HQNN.

IV. EXPERIMENT AND ANALYSIS

A. Data Preprocess

The experimental evaluation is based on the Car-Hacking dataset [13], which comprises five different categories: Normal, Gear Spoofing Attack, RPM Spoofing Attack, Fuzzy Attack, and DoS Attack. In the data preprocessing stage, the ID field of each message is encoded into 3 one-hot vectors, resulting in a 48-bit vector after encoding the 3-digit hexadecimal ID. Then combined with the DLC field, we obtain a 49-bit vector. A time window size of 64 is chosen, resulting in each training sample being represented as a 64×49 2D tensor. The label of the input data is binarized, with 0 representing attack data and 1 representing normal data. After preprocessing, a total of 7812 training samples are obtained. Then the dataset is split in a 9:1 ratio, resulting in the training set of 7030 samples and the test set of 782 samples.

B. Analysis of Detection Rate and Detection Efficiency

The most common evaluation metrics in the classification domain, including accuracy, precision, recall, ROC curve, and F1 score, are used to assess the performance of our model. Additionally, the model's inference efficiency is evaluated by repeatedly testing the top 5 samples in the test set 50 times and then calculating the average inference time. Fig.8 and Fig.9 present the heatmap of the confusion matrix and the ROC curve, respectively. The literature [12] achieves a high F1 score in the vehicular IDS using the same dataset. As a comparison, we take it as the baseline. The comparison of the model's detection results with other algorithms such as Decision Tree (DT), Random Forest (RF), XGBoost, Logistic Regression (LR) and Long Short-Term Memory (LSTM), CNN and Transformer can be found in Table I, which also includes the model's inference time for testing 5 data samples as a reference metric. As commonly observed, tree-based ML algorithms typically have simpler structures, resulting in shorter inference times, which are not specifically recorded in this experiment.

According to Table I, the proposed model demonstrates significant advantages over traditional tree-based machine learning algorithms, such as DT and RF, in terms of classification accuracy. This is attributed to the superior fitting capability of deep neural networks. Furthermore, compared to ML-based

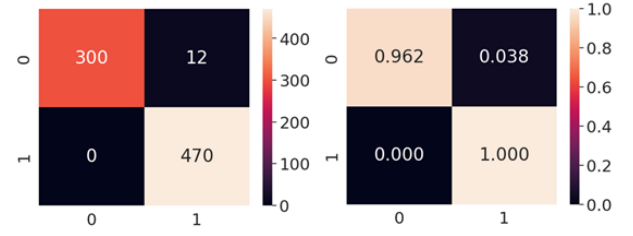


Fig. 8: Confusion matrix heatmap of detection results of the proposed model.

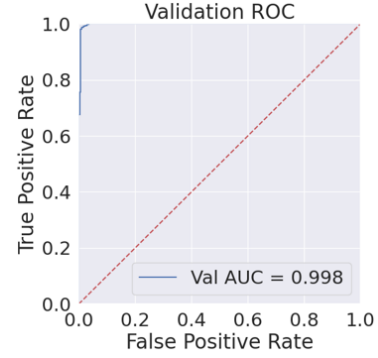


Fig. 9: ROC curve plot of detection results of the proposed model.

TABLE I: Comparison with other methods.

Method	Accuracy(%)	Recall(%)	Precision(%)	F1	Time(ms)
DT	88.11	86.14	83.65	0.8488	-
RF	94.12	98.54	86.54	0.9215	-
XGBoost	94.12	91.35	93.75	0.9253	-
LR	95.52	88.78	100.00	0.9406	-
LSTM	62.66	6.73	95.45	0.1257	278.9
CNN	82.10	55.13	100.00	0.7108	174.7
Transformer	93.09	82.69	100.00	0.9052	269.5
baseline [12]	100.00	100.00	100.00	1.0000	622.9
our method	98.47	96.15	100.00	0.9804	485.0

algorithms, the model outperforms LSTM, CNN and Transformer which are based on single architecture, indicating the effectiveness of the proposed model in accurately identifying intrusion behavior. Despite a slight difference in detection accuracy compared to the baseline model, the proposed model achieves remarkable progress in detection time, reducing inference time from 622.9 ms to 485.0 ms per 5 samples. With only a minor decrease of 0.0196 in F1 score compared to the baseline model, this advantage is deemed acceptable in practical engineering applications.

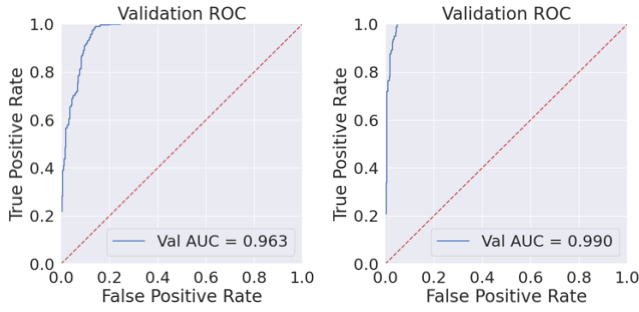
C. Analysis of Ablation Study

In the proposed model, Block serves as a core module. It encompasses various functional sub-modules. To dissect the specific contributions of each sub-module to the model's robustness and generalization capability, a series of ablation experiments are conducted. Under the premise of maintaining the unchanged architecture of the residual block, QNN layer,

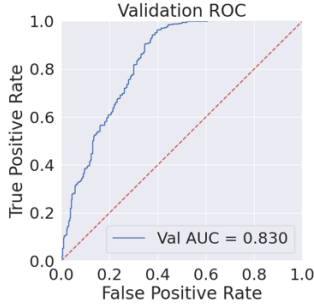
Fnet layer, and CNN layer are individually removed to observe their respective impacts on the final detection accuracy and inference efficiency. Fig.10a, Fig.10b and Fig.10c depict ROC curves of the model classification results after removing the QNN layer, Fnet layer, and CNN layer respectively. The comparative results of the model's detection performance after three sets of ablation experiments is shown in Table II.

TABLE II: Comparison of detection results after removing different layers in ablation study.

Ablation Module	Accuracy(%)	Recall(%)	Precision(%)	F1	Time(ms)
QNN	91.43	78.85	99.60	0.8801	180.7
Fnet	91.05	77.56	100.00	0.8736	223.2
CNN	81.20	55.13	96.09	0.7006	223.5



(a) ROC curve plot of model classification after removing the QNN layer from Block. (b) ROC curve plot of model classification after removing the Fnet layer from Block.



(c) ROC curve plot of model classification after removing the CNN layer from Block.

Fig. 10: ROC curves in ablation study.

Based on the data in Table II, it can be observed that removing different layers (QNN layer, Fnet layer, CNN layer) can have a significant impact on the model's performance. Specifically, after removing the QNN layer, the model's accuracy is 91.43%, with an F1 score of 0.8801. This indicates the critical role of the QNN layer in the model, being indispensable. Removal of the CNN layer has the most substantial impact on the model, reducing the F1 score from 0.9804 to 0.7006. This may be because removing the CNN layer hampers the model's ability to effectively utilize spatial information, thereby affecting its understanding of data

features and classification capability. In terms of efficiency, removing the QNN layer reduces the inference time for 5 sample data from 485.0 ms to 180.7 ms. This could be due to simulating QNN on classical computers, which increases the computational burden. Further experiments are needed to explore better methods for simulating QNN on classical computers and simplifying the structure of residual block to improve inference efficiency.

V. CONCLUSION

In this paper, we utilized the parallel computing capability of quantum computing proposing an HQNN-based vehicular IDS, which aims to enhance the efficiency and accuracy of intrusion detection in vehicular networks. Evaluation on the Car-Hacking dataset showed the HQNN-based IDS has made positive progress both theoretically and practically, providing a new solution for vehicular network security by combining the powerful capabilities of quantum computing and the flexibility of classical neural networks. Future work for the HQNN-based model can focus on optimizing the design of quantum circuits to reduce the number of layers and parameters, thereby lowering the computational complexity of the model and improving inference speed.

REFERENCES

- [1] M.-J. Kang and J.-W. Kang, "Intrusion detection system using deep neural network for in-vehicle network security," *PloS one*, vol. 11, no. 6, p. e0155781, 2016.
- [2] J. K. Kim, Q. Ho, S. Lee, X. Zheng, W. Dai, G. A. Gibson, and E. P. Xing, "Strads: A distributed framework for scheduled model parallel machine learning," in *Proceedings of the Eleventh European Conference on Computer Systems*, 2016, pp. 1–16.
- [3] Z. Yang and X. Zhang, "Entanglement-based quantum deep learning," *New Journal of Physics*, vol. 22, no. 3, p. 033041, 2020.
- [4] I. Cong, S. Choi, and M. D. Lukin, "Quantum convolutional neural networks," *Nature Physics*, vol. 15, no. 12, pp. 1273–1278, 2019.
- [5] G. Chen, Q. Chen, S. Long, W. Zhu, Z. Yuan, and Y. Wu, "Quantum convolutional neural network for image classification," *Pattern Analysis and Applications*, vol. 26, no. 2, pp. 655–667, 2023.
- [6] Y. Li, R.-G. Zhou, R. Xu, J. Luo, and W. Hu, "A quantum deep convolutional neural network for image recognition," *Quantum Science and Technology*, vol. 5, no. 4, p. 044003, 2020.
- [7] M. Henderson, S. Shakya, S. Pradhan, and T. Cook, "Quantum convolutional neural networks: powering image recognition with quantum circuits," *Quantum Machine Intelligence*, vol. 2, no. 1, p. 2, 2020.
- [8] S. Abdel-Khalek, M. Algarni, R. F. Mansour, D. Gupta, and M. Ilayaraja, "Quantum neural network-based multilabel image classification in high-resolution unmanned aerial vehicle imagery," *Soft Computing*, pp. 1–12, 2021.
- [9] S. Jeswal and S. Chakraverty, "Recent developments and applications in quantum neural network: A review," *Archives of Computational Methods in Engineering*, vol. 26, no. 4, pp. 793–807, 2019.
- [10] A. Ezhov and D. Ventura, "Quantum neural networks. future directions for intelligent systems and information sciences," *Studies in Fuzziness and Soft Computing*, vol. 45, 2000.
- [11] E. Grant, M. Benedetti, S. Cao, A. Hallam, J. Lockhart, V. Stojevic, A. G. Green, and S. Severini, "Hierarchical quantum classifiers," *npj Quantum Information*, vol. 4, no. 1, p. 65, 2018.
- [12] J. Zhou, S. Li, Y. Cao, H. J. Hadi, and H. Lin, "Robust intrusion detection system in can bus through multi-scale feature fusion," in *ICC 2024-IEEE International Conference on Communications*. IEEE, 2024.
- [13] H. M. Song, H. R. Kim, and H. K. Kim. (Mar. 13, 2023). Car Hacking Dataset. [Online]. Available: <https://ocslab.hksecurity.net/Datasets/CAN-intrusion-dataset>